重构协同：关于AI Native团队的思考

原创 天猫技术团队 大淘宝技术 2026年8月13日 20:12 浙江 71人

本文指出当前AI提效陷入“单点优化、全局低效”困境的根源在于协同缺失，主张通过重构生产侧将流程串联者从人转变为Agent，构建以“知识底座+Agent+人”为核心的AI Native团队闭环形态，其中人负责目标与判断，Agent基于统一的知识底座承担执行与串联；文章强调软件是被固化的知识，而实现该形态的关键瓶颈并非协同工具，而是如何为存量业务构建具备自治能力、能清晰定义业务规则与流程且可持续更新的知识底座，这已成为企业提升全局效率必须跨越的核心门槛。

01 写在前面

近一两年，公司内的AI项目可谓遍地开花：技术侧在探索各种AI Coding方案，业务侧在做业务运营的AI化。但热闹之下，似乎都撞上了一堵墙——单点上各有各的提效，全局上整体的效果却并不明显。

这是为什么？我的判断是：**公司真正的效率问题，从来不是单点的产能，而是“协同”；而这一年的AI提效，恰恰大多没碰到协同。** 这里举两种最典型的“拆碎”：

- 一种是**业务被横向切碎**。为了扛住系统和规模的复杂度，一个完整的业务被拆成平台、横向产品、垂直业务等一堆团队，各有各的KPI，目标并不一致。于是A业务GMV的增长，实质可能只是吃掉了B业务的份额，全局看甚至是负的——因为缺一个共同的全局目标。
- 一种是**产研流程被纵向拆碎**。需求、产品、研发、测试各管一段，中间靠人来回传递。而现在我们大多只盯着其中最小的一段——个人的AI Coding提效，写代码时“爽感”十足，但端到端算下来，整体效率的提升其实很有限。

诸如此类的“拆碎”叠加起来，就是“单点遍地提效、全局却不见增量、成本反而在涨”的根源。所以真正的破局，不是把某个环节再调快一点，而是**用“知识底座＋Agent”把协同这件事本身重做一遍**。

这篇文章不是要给出一套具体的解决方案，而是从这件事出发，讲讲我的理解。先说观点：一个AI Native团队能不能真正跑起来，核心落在**两件事**上——**建好知识底座，以及建好人、Agent、知识底座三者协同的工具。**

02 消费侧与生产侧

一切生产活动，最终都是为了满足人的需求。而人的原始需求本质是稳定的——马斯洛那张表几十年没怎么动过。所以变的从来不是需求本身，而是满足需求的方式。

顺着这一点，可以把所有事情切成两类。它们看起来都是“业务的一部分”，但在AI面前的命运并不相同：

- 一类直接面向人、满足人的需求——这是**消费侧**；
- 一类在背后组织和串联生产——这是**生产侧**。

先看消费侧。 人要看新闻，于是有了报纸；嫌报纸慢，有了门户网站；嫌守在电脑前麻烦，有了手机里的信息流；再往后，是AR眼镜，也许还有脑机接口。需求（“我想知道发生了什么”）从头到尾没变，变的只是那层交互——人和满足之间的界面。

到了AI时代，这一侧更像是朝“个人助理”的方向在走。钢铁侠里的贾维斯是我们可以想象的一种未来形态：它接管的是面向每个人“最后一公里”的需求，高度个性化——你要什么、想怎么要，是它来适配你，而不是你去适配产品。

所以消费侧的演进，本质是交互方式的一次次升级。但无论升级到哪一代，终端永远是人，“以人为本”在这一侧始终成立，不会被任何技术动摇。对我们这种互联网电商的公司来说，这一侧固然重要，但AI时代更多的机会和挑战，在另一侧。

再看生产侧。 也就是“B端”，它没有“终端是人”这个锚，完全是当下技术水平的产物——有什么样的技术，就长出什么样的流程、分工和系统。技术每升级一轮，这一侧就被重写一次：从手工作坊到流水线大生产，从纸质单据到ERP、OA，再到互联网化、移动化……每一次技术革命，都把“生产怎么组织”重做了一遍，我们这代人已经亲历过好几轮。

但这一次和以前不太一样的地方在哪呢？先看一下，一条业务流程，到底是靠什么“串”起来的？曾经的答案一直是——人。需求评审会上，是人把上下文从产品传给研发；工单系统里，是人把上一个环节的结论手动录进下一个环节；所谓“流程”，本质是一连串由人来完成的传递、判断和衔接，人的“泛化性”和“容错性”天生适合做串联。我们过去几十年开发的绝大多数生产侧软件，都在给“人来串联”这件事打辅助——表单是为了让人填，看板是为了让人看，审批流是为了让人点。它们共同的设计前提，是“人是串联者”。

AI时代最大的变化，就是一句：串联者从人，换成了Agent。

这也是“AI Native”和“AI辅助”的重要差异：

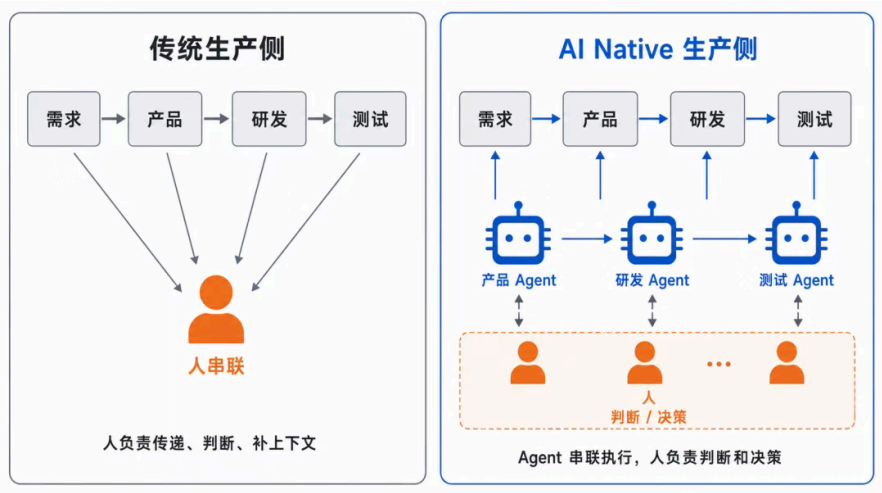
- **AI辅助**：流程骨架不动，串联者还是人，只是在某个环节塞一个AI功能，让人写得快一点、看得省一点，本质是在旧流程上加挂能力。
- **AI Native**：把串联者整个换掉，由Agent承担传递、判断和衔接，然后围绕“Agent是串联者”这个新前提，重建流程本身。

看清楚这一点，我们就明白，会消亡的不是“软件”这个笼统的概念，而是那些把“人是串联者”当作设计前提的软件——表单、报表、审批流，以及大量内部系统里“等人来操作下一步”的环节。它们要服务的那个“人”的角色，已经不在原来的位置上了。

举个身边的例子：运营侧大量的AI提效诉求，其实是取数、做日报周报，或者横向小二向行业小二“收作业”——这些工作，本质上都是在做信息的传递和协同。换个角度想，如果串联这件事本身交给了Agent，这些为“人传人”而存在的工作，也许压根就不必存在了。

换句话说，整个生产侧的形态，注定要被重构——不是在旧骨架上修修补补，而是围绕新的串联者重新长一遍。

观点总结：消费侧永远以人为本，变的只是交互；而生产侧的“串联者”正从人换成Agent——面向“人来串联”而开发的软件会消亡，整个形态必须重构。



03 AI Native的协同形态

把串联者换成Agent之后，一个团队理想上会长成什么样？

我能想象的是：以一个完整的业务单元为边界，里面大致有三层东西。

最底下，是一个统一的**知识底座**。它沉淀这个业务所有的业务规则、业务流程、领域知识和规范约束——也就是“这个业务究竟是怎么运转的”这件事本身。

中间，是一群**各司其职的Agent**：有做业务运营的、做产品和研发coding的、做测试的、做运维的、做客服的……它们都从知识底座取知识，各自承接并串联起自己环节的工作。当然，Agent光有知识还不够——它还得依赖一组**工具和技能**才能真正干活：查数据、调系统、跑流

程、触发变更。可以说，知识底座决定它“知道什么、该怎么做”，工具和技能决定它“能做到什么”。

> 为什么要拆成不同的Agent，而不是一个大而全的？因为不同Agent需要感知的知识范围、能提供的服务范围、被授予的权限和安全要求，往往都不一样；拆开之后，每个Agent的上下文相互隔离，评测和优化也能各自独立进行、独立迭代。举个例子：帮工程师排查线上问题的运维Agent，既要感知代码层面的知识，又要能调用各种运维工具（normandy、sunfire、arthas等）；面向内部员工的答疑Agent，可以直接回复到代码级的错误定位信息；而面向外部用户的客服Agent，要的则是更专业、更得体的客服话术。它们能看到的知识、能动的东西、出错的代价都不一样，自然不该、也不能糅成一个。

最上面，是**人**：业务、产品、技术等不同角色。关键是他们的位置变了——不再是流程里逐个传递信息的“接力棒选手”，而是和这群Agent一起，围绕同一个业务目标协同：定方向、做判断、处理Agent搞不定的例外，并把新产生的知识沉淀回底座。

这些角色之间的边界，也许会变模糊，现在已经有不少技术团队在做“全栈融合”的尝试，比如前后端不再分家。但要看清楚，这种融合目前主要发生在执行层面（比如写代码）；真正需要专业度的地方，差异不会完全抹平。说到底，一个角色的专业深度，决定了他在面对AI的产出时，能做哪一层的专业决策——当AI抛给你一套复杂的工程或架构方案，你得有能力判断它到底靠不靠谱、坑在哪里，而这种判断力未来会变得更加稀缺和重要，这不是“全栈”可以覆盖的。

这三层合起来，目标只有一个：让整个业务的运行，在“知识底座＋Agent＋人”之上形成闭环。和今天最大的差别在于——今天的串联靠人开会、拉群、手动同步，未来的串联靠Agent；今天的知识散在各人脑子里和一堆对不齐的文档里，未来的知识沉淀在一个能被Agent直接使用的底座里。

举一个具体的例子。

来了一个业务需求：不再是业务先写一份MRD给PD、PD转成PRD丢给研发、中间反复拉会对齐、研发再翻代码出方案；而是MRD、PRD的产出，本身就由业务和PD借助**业务/产品Agent**、基于知识底座协同完成，再由**研发Agent**基于知识底座，定位它涉及哪些领域、哪些应用、哪些规则，把技术方案拆解出来；人在一个协作空间里（比如项目群）与AI协同、澄清，并在关键节点做确认和判断；编码、测试、发布则由对应的Agent串起来执行；过程中新产生的决策和知识，再回流进底座。

同时，一个线上监控报警事件，触发了这个业务空间里的**运维Agent**的自动排查和定位，给出止血操作由人决策。

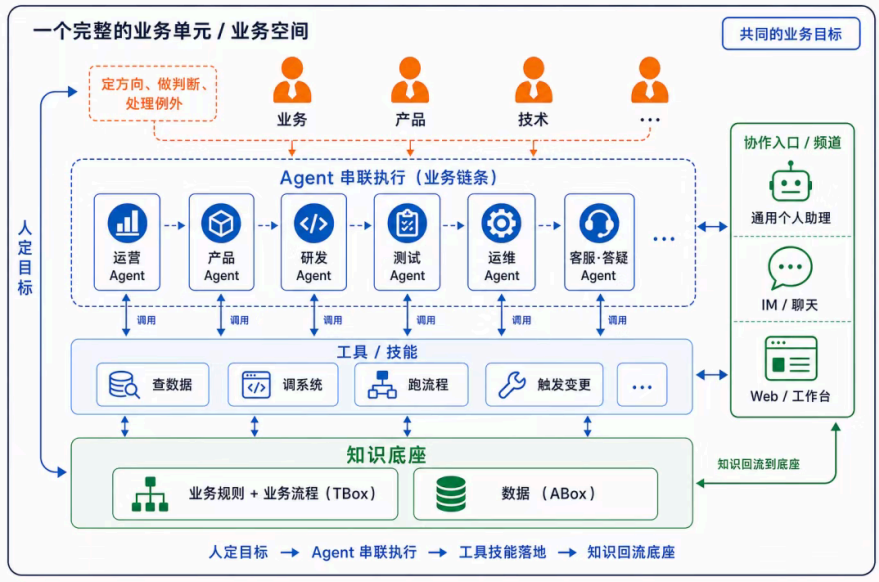
另外，当前业务空间的一个**答疑Agent**，基于知识底座自动回答用户的问题，并在不确定答案的情况下，通过团队内部群at项目成员进行确认，而后再回答给用户（参考Qteam团队的**Q仔**实践）；遇到用户提的bug，自动记录到Aone缺陷，并触发**研发Agent**自主拉变更修复。

同样，**运营Agent**基于知识底座，结合本业务的目标分析产出业务状况的报告，用于老板做重要的决策和判断，并给出运营动作等等。

在整个业务活动里面，人负责的是目标和判断，Agent承担的是串联和执行。

这个形态目前在公司内还没看到成熟的最佳实践，但一定有团队已经在探索了，至少方向是相对清楚的——“给现有流程配几个AI助手”必然达不到我们想要的结果。

观点总结：AI Native团队的理想形态，是以一个完整业务单元为边界的“知识底座＋Agent＋人”三层闭环——人定目标、做判断，Agent负责串联和执行。



04 顺带厘清：软件，是被固化的知识

这一节稍微岔开主线，把一个容易混淆的问题厘清——在上面这套形态里，我们熟悉的“软件”到底还算什么。往更抽象一层看，这套东西可以拆得很干净。

先看看知识底座里到底有什么。一个完整的业务闭环，至少包含两类东西：一类是这个业务“怎么运转”的定义——**业务规则加业务流程**；另一类是它运转起来之后产生的**数据**。

这很像本体论里的一个区分：业务规则和业务流程，是类型层面的定义（本体论里叫**TBox**）——规则是“是什么、要遵守什么”的陈述性知识，流程是“按什么次序做”的程序性知识，两者都是知识；而运转中产生的一笔笔订单、一个个商家和用户，是具体的事实（叫**ABox**）。一个完整的知识底座，应该同时包含TBox和ABox两部分。

那软件产品又是什么？在这个模型里，**它不是一个必需品**，就像现实世界业务本身可以脱离于软件存在一样。抽象地看，业务完全可以由Agent直接在知识底座上运转起来。软件之所以需要，是因为Agent的动态运转在某些地方还不够用——要交互体验、确定性、性能、成本、安

全、合规审计等等。于是我们把其中一部分业务规则和业务流程“固化”下来，做成**产品规则和产品流程**，这就落到了软件研发流程上（也包括AI Coding）。

换句话说，**软件是被固化的知识**。而“把知识固化成软件”这件事本身，又是一条业务流程——研发流程——它同样可以由Agent（Code Agent）在知识底座上运转完成。AI Coding，本质就是这个固化动作。

所以，前面说生产侧要被重构（面向人的GUI也许不再重要），但它不会被清空——那些为了确定性、性能、成本、合规而必须保留的“确定性逻辑”，最终就以产品规则和产品流程的形态，固化在软件这一层，而源头，始终是知识底座。

再往前看一步，“沉淀”这个动作本身也会变：**过去一个业务把经验沉淀下来，交付的是一套软件；往后，交付的更多会是“知识”（也包括Skill）；过去做集成是交付一个端到端的软件系统，往后更多是给Agent交付一件件工具。**

05 两件事：知识底座是瓶颈，协同工具是基建

回到开头说的那两件事——要让上面这套形态真正跑起来，公司要面对的核心命题就这两件事：一是**把知识底座建起来**；二是**把“知识底座、人、Agent三者协同”的工具建起来**。

先说协同工具。长期看我认为它不太会是瓶颈，但它非常重要——而且它是通用的基础能力，理论上跨团队协同需要无缝，因此它不适合由各业务自己重复造，但当下基础团队也并没有一个比较完整的方案可用，业务侧团队在选型上存在困难，有时候不得不自建。

事实上，公司内已经有不少团队和产品在往这个方向走：比如Aone团队的Devix、Qteam团队的Qoder系列（Qoder、QoderWake等），以及企业智能的数字员工账号、ideawork等等，但都还很不完善，属于探索阶段。我认为，现在很多号称“数字员工”的方案，应该跳出数字员工本身，核心把重心放在Agent和人的协同形态上。

从Devix、Qoder系列、ideawork等等，再到一些团队内部探索的产品，其实已经能看到协同工具该有的一些关键概念雏形：

- **业务空间**：划定一个业务闭环边界的核心容器（Devix的产品空间类似）；
- **参与者**：空间里的人类成员和各类Agent（概念上类似大家说的数字员工），关联技能和知识；
- **任务**：对应一个项目、一个需求、或一个讨论的话题；
- **频道**：人与Agent协同的入口，IM、Web，甚至很可能是QoderWork这样的通用助手（**传统协同入口在钉钉，未来也许在通用助手**）；
- **产物**：协同过程中沉淀下来的结果；
- **知识底座**：而这是最重要的，每个业务空间管理的知识+数据，这是业务的护城河。

这里要多说一句，因为它常被误解：企业级业务真正的复杂度和瓶颈，往往不在写代码这一段。拿半托管超链这类跨团队项目来说，光是把商业逻辑、方案取舍在多个角色之间讨论清楚，就极其耗时——这和“把某段代码写快点”完全是两码事。所以别被“Coding变快了”制造的幻觉带偏：简单需求哪怕让AI全自动交付，对全局的提升也有限；甚至我们现在常干的一件事——研发为了“完美地写代码”，反过来要求PD给出更完整、更规范的PRD——本身就是把产品设计和研发割裂开了，中间那道协同成本一点没省。

也正因如此，这些协同工具不该只停在研发流程里（AI Coding），真正的AI Native协同，应该要往更左侧的需求侧、业务侧走。但不管怎样，协同工具“能不能做出来”，方向是清楚的，剩下更多是时间问题。

真正难的，是另一件事——知识底座。它不通用、不可复制，每一块业务都得自己长出一套来，而且做得越久的传统业务，这件事越难。

观点总结： 协同工具未来不会是瓶颈，当下的关键是把理想的协同形态探索清楚；而真正长期卡住全局的，是能不能构建起一个个完整业务闭环的知识底座。



最难的一关：存量业务的知识底座

对新业务、新团队，知识底座并没那么难。一张白纸，可以从第一天就按“知识底座＋Agent＋人”的方式来组织，规则、流程、数据一开始就往底座里沉，不用还历史的债。

真正的硬骨头是**存量业务**。

要看清它难在哪，先看看里面可能有哪些东西：

分类	内容	事实来源	落地难度
业务知识	商业模式、目标、行业与市场、业务规则、业务流程（这个业务到底怎么跑），重要业务方向的决策背景等等	大多在业务/PD脑子里，部分在战略与业务文档里，还有些在会议纪要、聊天记录里	最难：高度隐性、易变
产品知识	领域模型（商品－订单－会员等实体与关系）、产品逻辑、产品流程/工作流、规范约束——本质是业务被“固化”成产品的逻辑视图	理想应锚定代码和配置（同技术知识），但也有很多在产品文档、PRD等文档中	中：部分同技术知识，来自文档的部分取决于保鲜程度
技术知识	系统架构、应用与服务、API、数据模型与字段含义、技术规范与基建——它是产品知识的物理视图	代码仓库、部署信息、线上配置（数据库/diamond/switch等等）	较易：可从代码和配置、基建服务自动抽取
运行数据	订单、商家、用户、库存、各类指标等真实实例	数据库、数仓、日志、监控	最易：天然在线、可自动采集
经验知识	故障复盘、排查与踩坑、最佳实践、例外处理	工单、复盘记录、值班群，大量在人脑	难：隐性、散落

这些分类因事实来源不同，落地难度也不同，因此我们需要尽量把每条知识**锚定到一个权威的事实来源**，凡是能从代码、配置、数据里自动生成和校验的，就绝不靠人肉维护；越靠人脑那

端的，越要专门设计沉淀机制去外化，而**能不能自治，取决于知识离权威来源有多近。**

存量业务的难，也就藏在这张表里，具体有三个坎：

第一，难在知识底座的构建过程和方法。 存量业务的系统和架构是历史一块块长出来的，边界、耦合、字段含义里都还留着当年的痕迹。在一个划定的范围里，把知识底座扎扎实实地构建出来——这个构建的过程和方法本身，才是最难、也是最有价值的部分。因为它做的事，是抛开历史包袱、重新把“这个业务到底怎么运转”讲清楚一遍；而这个结果，理论上应该能反过来指导组织和架构该长成什么样。

第二，大量知识根本不在系统里，在人脑子里。 一个做了多年的业务，最关键的那些规则、那些“为什么当年这么定”的来龙去脉、那些只有老人才清楚的例外，大多是隐性的，散在一个个人的脑子里和对不齐的文档中。怎么把它们沉淀、结构化、外化成底座，是最费劲的部分。（这其实不是AI时代才有的痛——为什么文档总是脱离代码、架构大图总是反映不了最新状态？这个老问题，AI不会自动替你解决。）

第三，底座必须能“自己活下去”。 业务天天在变，知识也跟着变。如果底座靠人肉更新维护，历史已经无数次证明，它很快就会烂掉、过期、没人再信。所以它必须具备自治能力——靠自动化加人工确认来保鲜，而不是靠哪位架构师的责任心硬扛。

本体论[9]也许是个方向，它至少提供了一套描述“规则、流程、数据及其关系”的语言。但落地真正难的，还是怎么在一个具体的、千头万绪的存量业务里，把它建起来、并且持续转下去。

现状是，越来越多团队已经意识到这件事的分量，但也正因为它太难，很多时候被绕开了——大家在原有的组织框架里，用各自割裂的方式零敲碎打，谁也不愿意去碰那个最硬的核。

还有一个问题，需要摆出来：**知识底座到底该以多大的范围为单位来建、来闭环？** 单元划得太大，规则和流程千头万绪、相互耦合，底座很难收敛，建到一半就陷在复杂度里；划得太小，又凑不成一个完整的业务闭环，Agent一串联就频繁跨出边界，结果又退回到“靠人去对接”的老路。这个边界到底划在哪儿最合适？

这件事其实非常重要——它还关乎组织结构，比如一个业务的需求，若实际依赖商品详情、交易等平台团队，就没法只从“我这个业务”的视角把它闭环掉。两年前架构组进行了大量关于业务和产品、业务大图、产品树等概念和方案的讨论，当时没有一个定论，但我觉得，那时在“非AI”背景下讨论的这些东西，和我们现在想要的这个知识底座本质就是一回事，而在AI的背景下，有**更大必要性和可行性**了。

07 最后：一道绕不开的选择题

其实，把一个业务彻底讲清楚、让它可被复用——这件事一直都很难，并不是AI才带来的。只是过去这么多年，因为串联者是人，这个难题被人“扛”过去了：知识不全、流程不清、文档过期，靠的是一个个人在中间补位、对接、救火，谈不上高效，但业务照样转，问题也就被一直掩盖着。

到了AI时代，串联者从人换成Agent，这块遮羞布被揭开了。Agent没法像人那样靠经验和默契去补位，它只能在知识底座上运转——底座有多清楚，它就能跑多远。于是“把知识底座建起来”这件事，从一件可以一直拖着的事，变成了绕不开的必选项。

如果一家大公司迈不过这道坎，迟早会有没有历史包袱的创业公司，用“知识底座＋Agent”跑出数倍的效能，把它甩在身后。创业公司的优势不在更聪明，而在它是张白纸——前面那些存量之难，它一样都不用还，整个AI Native的形态一旦跑起来，数据和知识的飞轮就自然转动起来了。

其实公司里效率问题的根源，这么多年一直没变——业务、产品、组织怎么真正配位，怎么对齐到一个共同目标、减少内耗、降低协同成本。技术架构的目标也没变过：从机器码到汇编、到高级语言，再到领域驱动设计、上云、Serverless……每一步，都是为了把人从琐碎里解放出来，更多地聚焦在创新和业务逻辑本身。AI只是把这条线又往前推了一大步。而这一步能不能真正走通，最终还是取决于：我们能不能把那个属于自己业务的知识底座，扎扎实实地建起来。



本文作者书牧，来自淘天集团–天猫技术团队。天猫技术是阿里巴巴旗下专注于电商场景的综合技术团队，不仅服务于自身电商业务增长，也在不断探索如何用AI、大数据、交互技术赋能行业交易、品牌营销与消费者体验，成为全球领先的品质购物平台和技术引擎。

拓展阅读

3DXR技术 | 终端技术 | 音视频技术

服务端技术 | 技术质量 | 数据算法

大淘宝技术
大淘宝技术官方账号
966篇原创内容
公众号

阅读 1.4万

留言 2

写留言

Doge的万事屋 浙江 8月14日 1

AI 大灌水时代难得的好文

漫画里的狄... 广东 8月13日

我目前在企业内部搭建 知识底座，采用的开源的上下文知识库ov，发现企业级别内部 权限管控这类 还没做的很细，希望后续的文章可以出一下ABox和TBox在产生的过程中如果设置 人、agent两个维度的 权限范围和管控，以及阿里目前有开源的知识底座项目吗，感谢分享，对AI Native的方向基础有个更扎实的理论基础。